

实验报告四

姓名： 李祥熙 学号： 2234412799

目录

1 实验目的	2
2 实验过程	2
2.1 实验环境	2
2.2 数据集与预处理	2
2.3 实现步骤	3
3 实验内容	3
3.1 模拟器设计思想	3
3.2 模拟器特色	4
3.3 实验参数组合	4
4 实验结果	5
4.1 各 trace 的访存特征	5
4.2 某一具体配置的详细统计	5
4.3 Cache 大小对失效率的影响	6
4.4 相联度对失效率的影响	7
4.5 块大小对失效率的影响	7
5 总结	8

1 实验目的

本实验的目标如下：

- 加深对 Cache 大小、相联度、块大小、替换算法等因素对 Cache 性能影响的理解；
- 自行设计并实现一个可配置的 Cache 模拟器（模拟器 A）；
- 使用真实程序的访存地址流（trace）对模拟器进行测试；
- 通过控制变量的方式，分别考察各个参数对失效率的影响，并给出分析结论。

2 实验过程

2.1 实验环境

- 操作系统：macOS (Darwin 25.5.0)
- 编程语言：C++ (C++17 标准)
- 编译器：g++ -O2

本实验选择模拟器 A，即完全自行实现 Cache 模拟器。选择 C++ 是因为每个 trace 文件约有 100 万条记录，而完整实验需要遍历 256 种参数组合，C++ 的执行效率可以保证整套实验在数十秒内完成。

2.2 数据集与预处理

实验使用本次实验提供的 4 个 trace 文件，它们是 SPEC 基准程序的访存地址流：

- 022.li.din: LISP 解释器，约 100 万条记录；
- 047.tomcatv.din: 向量化的网格生成程序，访存以数组为主；
- 078.swm256.din: 浅水方程数值模拟，以读为主；
- 085.gcc.din: GCC 编译器，访存模式不规则。

每个 trace 文件中，每一行的格式为 `access_type address [size/data]`，其中 `access_type` 取值含义为：0 表示 load data（读），1 表示 store data（写），2 表示 fetch instruction（取指）；`address` 为 16 进制表示的 32 位地址。一个典型的片段如下：

```
2 400190 8fa40000
0 7ffe6c64 0
2 400194 3c1c1001
1 100039b8 0
```

按照实验要求 H (“为简化模拟器实现，仅考虑 load data 和 store data”)，模拟器在读取地址流时 **只处理类型 0 和类型 1 的访存记录，忽略类型 2 的取指记录**。每次访存按 4 字节处理。

2.3 实现步骤

1. 设计 Cache 的数据结构：将 Cache 组织为“组—路—行”的三级结构，每行只保存 tag 和有效位；
2. 实现地址划分逻辑：根据块大小和组数，把访存地址拆分为 tag、组索引、块内偏移三部分；
3. 实现 LRU 替换算法：每组内的若干路按最近使用顺序排列，命中或换入时把该行移到最前；
4. 实现写分配 (write-allocate) 策略：写失效时也把数据块调入 Cache；
5. 实现命令行参数解析，支持 trace_input_file、cache_size、cache_associativity、cache_block_size 四个参数；
6. 统计并输出读/写访存数、读/写失效数、替换块数以及各项失效率；
7. 编写批处理脚本，遍历全部参数组合，把结果汇总成 CSV 文件用于分析。

3 实验内容

3.1 模拟器设计思想

模拟器采用经典的**组相联 Cache**模型，这是直接映射（相联度为 1）和全相联（组数为 1）的统一形式。其核心参数关系为：

$$\text{块数} = \frac{\text{cache_size}}{\text{block_size}} \quad (1)$$

$$\text{组数} = \frac{\text{块数}}{\text{cache_associativity}} \quad (2)$$

对于一个访存地址，去掉低位的**块内偏移**后得到块地址；块地址的低位作为**组索引**，用于定位到具体某一组；剩余的高位作为 **tag**，用于在组内比较是否命中。由于 cache_size、block_size、assoc 均为 2 的幂，组数也必然是 2 的幂，因此组索引可以用一次按位与 (& (numSets-1)) 高效计算。地址划分的核心代码如下：

```
uint64_t blockAddr = addr >> offsetBits_;
uint64_t index = numSets_ > 1 ? (blockAddr & (numSets_ - 1)) : 0;
uint64_t tag    = blockAddr >> indexBits_;
```

LRU 替换算法的实现采用了一个简洁的技巧：把每一组的各路保存在一个向量中，**始终让最近使用的行排在最前面（front = MRU），最久未使用的行排在最后（back = LRU）**。这样，命中时只需把命中行移到队首；失效换入时，若该组已满（最后一行有效）则发生一次替换，直接弹出队尾、把新行插入队首即可。由于相联度最多为 8，这种线性操作的开销可以忽略。单次访存处理的核心逻辑如下：

```
auto& set = sets_[index];
for (size_t i = 0; i < set.size(); ++i) {
    if (set[i].valid && set[i].tag == tag) {    // ---- 命中 ----
        moveToFront(set, i);                  // 更新 LRU 顺序
        return;
    }
}
// ---- 失效 ---- (写分配：失效总是把块调入 Cache)
if (isWrite) ++writeMisses_; else ++readMisses_;
if (set.back().valid) ++replacements_;        // 该组已满，发生替换
set.pop_back();
set.insert(set.begin(), Line{tag, true});      // 新块插入队首
```

3.2 模拟器特色

- **统一的组相联模型**：相联度为 1 时即直接映射，组数为 1 时即全相联，一套代码覆盖全部情况；
- **参数合法性检查**：要求三个参数均为 2 的幂，并校验 $\text{block_size} \leq \text{cache_size}$ 以及 $(\text{cache_size}/\text{block_size}) \% \text{assoc} == 0$ ，避免非法配置；
- **友好的参数输入**：大小参数支持 K/M 后缀，例如 `--cache_size 32K`；
- **两种输出模式**：默认输出可读的统计表格，加 `--csv` 则输出单行机器可读数据，便于批量汇总；
- **流式读取**：逐行处理 trace，内存占用与文件大小无关。

3.3 实验参数组合

按照实验要求，固定使用 **LRU 替换算法**和**写分配策略**，对以下参数进行充分测试：

表 1: 实验参数取值范围

参数	取值
trace 文件	022.li、047.tomcatv、078.swm256、085.gcc
Cache 大小	8KB、16KB、32KB、64KB
相联度	1、2、4、8
块大小	16B、32B、64B、128B

四个维度各取 4 个值,共 $4 \times 4 \times 4 \times 4 = 256$ 种组合,全部测试结果汇总在 `results/results.csv` 中。下文通过**控制变量法**,每次只改变一个参数来观察其影响。

4 实验结果

4.1 各 trace 的访存特征

首先给出各 trace 在仅统计 load/store 后的访存规模,这有助于理解后面失效率的差异。

表 2: 各 trace 的访存特征 (仅 load/store)

trace	读次数	写次数	写占比
022.li	155399	102350	39.7%
047.tomcatv	253723	130733	34.0%
078.swm256	192459	9894	4.9%
085.gcc	139816	80669	36.6%

可以看到 078.swm256 的写操作占比极低 (仅 4.9%),是典型的“读密集”程序;其余三个程序读写比例较为均衡。这一特征会直接影响它们对写失效的敏感程度。

4.2 某一具体配置的详细统计

为说明模拟器输出的完整内容,下面给出在 **Cache=32KB、相联度=4、块大小=64B** 这一组配置下,对 022.li 运行得到的原始输出 (即默认表格模式):

```
===== Cache Simulation Result =====
Trace file      : trace files/022.li.din
Cache size      : 32768 B (32 KB)
Block size      : 64 B
Associativity    : 4-way
Number of sets   : 128
Replacement     : LRU      Write-miss: write-allocate
```

Data accesses	: 257749 (read 155399, write 102350)
Read misses	: 452
Write misses	: 872
Replaced blocks	: 812
Read miss rate	: 0.2909 %
Write miss rate	: 0.8520 %
Total miss rate	: 0.5137 %
=====	

该输出包含了实验要求的全部统计项：读/写访存数、读/写失效数、替换块数，以及读、写和总的失效率。其中“Number of sets = 128”由 $32768/64/4 = 128$ 计算得到；“Replaced blocks”小于失效总数 ($452 + 872 = 1324$)，差值 $1324 - 812 = 512$ 正是各组首次被填满之前的**冷启动失效**（强制失效），它们不引起替换。

把该配置推广到全部 4 个 trace，得到表 3 的横向对比：

表 3: Cache=32KB、相联度 =4、块大小 =64B 下的详细统计

trace	读失效	写失效	替换块数	读失效率	写失效率
022.li	452	872	812	0.29%	0.85%
047.tomcatv	168	8573	8229	0.07%	6.56%
078.swm256	33	526	57	0.02%	5.32%
085.gcc	683	1330	1501	0.49%	1.65%

表 3 揭示了一个有趣现象：047.tomcatv 和 078.swm256 的**读失效率极低但写失效率较高**。这是因为它们是数值计算程序，对数组的读取具有很好的空间局部性（块被读入后被反复读取），而写操作往往是“先写后才会再用”的初始化式写入，在写分配策略下每次写新块都会产生一次写失效。

4.3 Cache 大小对失效率的影响

固定**相联度 =4、块大小 =64B**，改变 Cache 大小，总失效率（%）如表 4 所示。

表 4: Cache 大小对总失效率的影响（相联度 =4，块大小 =64B，单位：%）

trace	8KB	16KB	32KB	64KB
022.li	1.4180	0.7845	0.5137	0.3845
047.tomcatv	2.6461	2.2744	2.2736	2.2653
078.swm256	0.2762	0.2762	0.2762	0.2762
085.gcc	2.8696	1.2055	0.9130	0.7597

结果分析：

- **整体趋势是 Cache 越大失效率越低**，这符合理论预期：容量增大减少了容量失效；
- 022.li 和 085.gcc 受容量影响最明显（085.gcc 从 8KB 的 2.87% 降到 64KB 的 0.76%），说明它们的工作集较大、对容量较敏感；
- 047.tomcatv 在 16KB 之后几乎不再下降，078.swm256 则**从头到尾保持 0.2762% 不变**。这是因为它们剩下的失效几乎都是**强制失效**（每个数据块第一次访问必然失效），而强制失效与 Cache 大小无关，因此增大容量无济于事。

4.4 相联度对失效率的影响

固定 Cache=16KB、块大小 =64B，改变相联度，总失效率（%）如表 5 所示。

表 5: 相联度对总失效率的影响（Cache=16KB，块大小 =64B，单位：%）

trace	1 路	2 路	4 路	8 路
022.li	2.2184	0.9432	0.7845	0.7333
047.tomcatv	3.0050	2.3815	2.2744	2.2827
078.swm256	0.3889	0.2762	0.2762	0.2762
085.gcc	3.1671	1.5248	1.2055	1.0985

结果分析：

- **提高相联度能降低失效率**，因为它减少了多个地址映射到同一组而互相驱逐造成的**冲突失效**；
- **收益主要集中在从 1 路到 2 路的阶段**：022.li 从 2.22% 骤降到 0.94%，085.gcc 从 3.17% 降到 1.52%，几乎减半；
- 而从 4 路提高到 8 路时改善很小，047.tomcatv 甚至略有回升（2.2744%→2.2827%）。这说明当冲突失效已被基本消除后，继续提高相联度的边际收益递减，剩下的失效以强制失效为主；
- 这一规律与课堂上的结论一致：“**2 路组相联 Cache 大致相当于容量翻倍的直接映射 Cache**”，即提高相联度和增大容量在降低冲突失效上有相似效果。

4.5 块大小对失效率的影响

固定 Cache=32KB、相联度 =4，改变块大小，总失效率（%）如表 6 所示。

表 6: 块大小对总失效率的影响 (Cache=32KB, 相联度 =4, 单位: %)

trace	16B	32B	64B	128B
022.li	1.6729	0.9327	0.5137	0.2863
047.tomcatv	8.6767	4.4073	2.2736	1.2051
078.swm256	1.0546	0.5387	0.2762	0.1428
085.gcc	2.6437	1.5058	0.9130	0.5710

结果分析:

- 在本实验的取值范围内, 增大块大小对降低失效率效果非常显著, 几乎每翻一倍块大小, 失效率就大致减半;
- 这是因为更大的块一次调入更多相邻数据, 充分利用了程序的空间局部性, 从而减少了强制失效的次数;
- 数值计算程序 047.tomcatv 的改善最为剧烈 (从 16B 的 8.68% 降到 128B 的 1.21%), 因为它顺序访问数组, 空间局部性极强, 大块能一次性覆盖更多后续访问;
- 需要说明的是, 块大小并非越大越好: 块过大时会减少 Cache 中的总块数, 可能引入更多冲突失效, 并增大每次失效的传输开销。本实验最大块为 128B, 尚未触及这一拐点, 所以表现为单调下降。

5 总结

本实验自行设计并实现了一个组相联 Cache 模拟器 (模拟器 A), 采用 LRU 替换和写分配策略, 支持 `trace_input_file`、`cache_size`、`cache_associativity`、`cache_block_size` 四个参数, 并用 4 个真实程序的 trace 进行了 256 组组合测试。主要完成内容与结论如下:

- 实现了正确、高效的 Cache 模拟器, 一套代码统一覆盖直接映射、组相联和全相联;
- 增大 Cache 容量主要减少容量失效, 对工作集大的程序 (gcc、li) 效果好, 但无法消除强制失效;
- 提高相联度主要减少冲突失效, 收益集中在 1 路到 2 路阶段, 之后边际递减;
- 增大块大小通过利用空间局部性显著减少强制失效, 对顺序访存的数值程序 (tomcatv) 效果最突出;
- 三类参数的效果与三种失效 (强制、容量、冲突) 的成因一一对应, 实验数据与课堂理论高度吻合。

实验感悟：通过亲手实现模拟器，我对 Cache 的工作原理有了远比看公式更直观的理解。尤其是观察到“增大容量对某些程序毫无作用、而增大块大小却效果惊人”这类现象时，真正体会到 Cache 性能优化必须结合具体程序的访存模式（局部性）来权衡，不存在对所有程序都最优的单一配置。控制变量法在分析多参数系统时也非常有效，它让每个参数的独立影响清晰可见。

附录：实验源代码

A.1 主要源码：src/cache_sim.cpp

```
// 配置化的组相联 Cache 模拟器（模拟器 A）
// 替换策略：LRU          写失效策略：写分配（write-allocate）
// 仅模拟 load(0) 和 store(1)，忽略取指(2)

#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>
#include <fstream>

struct Line { uint64_t tag = 0; bool valid = false; };

class Cache {
public:
    Cache(uint64_t cacheSize, uint32_t blockSize, uint32_t assoc)
        : blockSize_(blockSize), assoc_(assoc) {
        numBlocks_ = cacheSize / blockSize;
        numSets_ = numBlocks_ / assoc;
        offsetBits_ = log2u(blockSize);
        indexBits_ = log2u(numSets_);
        sets_.assign(numSets_, std::vector<Line>(assoc)); // front=MRU
    }

    void access(uint64_t addr, bool isWrite) {
        uint64_t blockAddr = addr >> offsetBits_;
        uint64_t index = numSets_ > 1 ? (blockAddr & (numSets_ - 1)) :
            0;
        uint64_t tag = blockAddr >> indexBits_;
        if (isWrite) ++writes_; else ++reads_;
        auto& set = sets_[index];
        for (size_t i = 0; i < set.size(); ++i) {
```

```

        if (set[i].valid && set[i].tag == tag) {    // 命中
            moveToFront(set, i); return;
        }
    }
    if (isWrite) ++writeMisses_; else ++readMisses_; // 失效
    if (set.back().valid) ++replacements_;          // 满组->替换
    set.pop_back();
    set.insert(set.begin(), Line{tag, true});        // 写分配
}

uint64_t reads() const { return reads_; }
uint64_t writes() const { return writes_; }
uint64_t readMisses() const { return readMisses_; }
uint64_t writeMisses() const { return writeMisses_; }
uint64_t replacements() const { return replacements_; }
uint64_t numSets() const { return numSets_; }

private:
    static uint32_t log2u(uint64_t x) { uint32_t b=0; while(x>1){x
        >=>1; ++b;} return b; }
    static void moveToFront(std::vector<Line>& set, size_t i) {
        Line tmp = set[i]; set.erase(set.begin()+i); set.insert(set.
            begin(), tmp);
    }
    uint32_t blockSize_, assoc_;
    uint64_t numBlocks_, numSets_;
    uint32_t offsetBits_, indexBits_;
    std::vector<std::vector<Line>> sets_;
    uint64_t reads_=0, writes_=0, readMisses_=0, writeMisses_=0,
        replacements_=0;
};

static bool isPow2(uint64_t x){ return x && !(x&(x-1)); }
static uint64_t parseSize(const std::string& s){    // 支持 K/M 后缀
    char suf = s.empty()?0:s.back(); uint64_t mul=1; std::string num=s;
    if(suf=='K' || suf=='k'){mul=1024ULL; num.pop_back();}
    else if(suf=='M' || suf=='m'){mul=1024ULL*1024; num.pop_back();}
    return std::strtoull(num.c_str(), nullptr, 10)*mul;
}

int main(int argc, char** argv){
    std::string tracePath; uint64_t cacheSize=0, blockSize=0; uint32_t
        assoc=0;

```

```

bool csv=false;
for(int i=1;i<argc;++i){
    std::string a=argv[i];
    auto next=[&]()->std::string{ if(i+1>=argc) std::exit(1);
        return argv[++i]; };
    if(a=="--trace") tracePath=next();
    else if(a=="--cache_size") cacheSize=parseSize(next());
    else if(a=="--assoc") assoc=(uint32_t)std::strtoul(next().c_str(),
        nullptr,10);
    else if(a=="--block_size") blockSize=parseSize(next());
    else if(a=="--csv") csv=true;
}

if(tracePath.empty()||!cacheSize||!blockSize||!assoc) return 1;
if(!isPow2(cacheSize)||!isPow2(blockSize)||!isPow2(assoc)) return
    1;
if(blockSize>cacheSize||(cacheSize/blockSize)%assoc!=0) return 1;

std::ifstream in(tracePath); if(!in) return 1;
Cache cache(cacheSize,(uint32_t)blockSize,assoc);
int type; std::string addrHex, rest;
while(in >> type >> addrHex){
    std::getline(in, rest); // 丢弃 size/data 字段
    if(type==0) cache.access(std::strtoull(addrHex.c_str(),
        nullptr,16), false);
    else if(type==1) cache.access(std::strtoull(addrHex.c_str(),
        nullptr,16), true);
    // type==2 取指：忽略
}

uint64_t reads=cache.reads(), writes=cache.writes();
uint64_t rMiss=cache.readMisses(), wMiss=cache.writeMisses();
uint64_t accesses=reads+writes, misses=rMiss+wMiss;
auto rate=[](uint64_t m,uint64_t n){ return n?100.0*(double)m/(
    double)n:0.0; };
if(csv){
    std::printf("%s,%llu,%u,%llu,%llu,%llu,%llu,%llu,%.4f,%.4f
        ,%.4f\n",
        tracePath.c_str(),(unsigned long long)cacheSize,assoc,(
        unsigned long long)blockSize,
        (unsigned long long)reads,(unsigned long long)writes,
        (unsigned long long)rMiss,(unsigned long long)wMiss,
        (unsigned long long)cache.replacements(),

```

```

        rate(rMiss,reads),rate(wMiss,writes),rate(misses,accesses))
        ;
    } else {
        std::printf(" Data accesses : %llu (read %llu, write %llu)\n",
            (unsigned long long)accesses,(unsigned long long)reads,(
                unsigned long long)writes);
        std::printf(" Read miss %.4f%%  Write miss %.4f%%  Total %.4f
            %%\n",
            rate(rMiss,reads),rate(wMiss,writes),rate(misses,accesses))
            ;
    }
    return 0;
}

```

A.2 编译与运行命令

编译

```
g++ -O2 -std=c++17 -o cache_sim src/cache_sim.cpp
```

运行单个配置（参数：trace 文件、Cache 大小、相联度、块大小）

```
./cache_sim --trace "trace files/022.li.din" \
    --cache_size 32K --assoc 4 --block_size 64
```

批量运行全部 256 种组合，结果写入 results/results.csv

```
./run_experiments.sh
```

A.3 批处理脚本核心：run_experiments.sh

```

TRACES=("022.li.din" "047.tomcatv.din" "078.swm256.din" "085.gcc.din")
SIZES=("8K" "16K" "32K" "64K")
ASSOCS=(1 2 4 8)
BLOCKS=(16 32 64 128)
for t in "${TRACES[@]}; do
    for s in "${SIZES[@]}; do
        for a in "${ASSOCS[@]}; do
            for b in "${BLOCKS[@]}; do
                ./cache_sim --trace "trace files/$t" --cache_size "$s" \
                    --assoc "$a" --block_size "$b" --csv >> results/
                    results.csv
            done
        done
    done
done

```

done
done
done